

LLM4MentalHealth Documentation

Gavin Bayley

August 2026



Contents

1	Assembly Instructions	3
1.1	Flower Head	3
1.2	Stem	5
1.3	Flower Pot	6
1.3.1	Servo Assembly	6
1.3.2	Plates Assembly	7
1.3.3	Electronics Plate	9
1.3.4	Assembling the Assemblies	9
1.4	Assembling the Assemblies	10
1.4.1	Connecting the Flower Head with the Stem	10
1.4.2	Stringing the Flower Pot and Final assembly	11
2	Repairs & Tuning	12
2.1	Tensioning the Tendons	12
2.2	Re-stringing Tendons	13
3	Software Overview	14
3.1	Communication	14
3.2	Architecture at a Glance	15
3.3	ROS 2 Topics Reference	15
3.4	Repository Layout	16
3.5	Embedded Firmware (ESP32)	17
3.6	Host Software Stack	18
3.7	Wi-Fi / micro-ROS Connection Protocol	18
4	Host Setup	19
4.1	Supported Configuration	19
4.2	Required Software & Dependencies	19
4.3	Cloning & Building the ROS 2 Workspace	20
4.4	Flashing the ESP32 Firmware	21
5	Startup Instructions	21
5.1	Every-Day Startup	22
5.2	First-Time Startup on a New Flower Unit or New Wifi	22
5.3	Connecting to a Different Wi-Fi Network	22
5.4	LED Status Indicator & Troubleshooting	22
5.5	Shutting Down	23
5.6	Quick Command Reference	24
A	Bill of Materials	24
B	CAD & Fabrication Source Files	34
B.1	Fusion 360 Models	34
B.2	Laser-Cut Sketches	34
B.3	Bill of Materials Source	34

1 Assembly Instructions

This section walks through assembly of the flower, from the flower head and stem down to the flower pot that houses the electronics and drive mechanism. Before starting any sub-assembly below, confirm the required parts are on hand against the Bill of Materials in Appendix A.

1.1 Flower Head

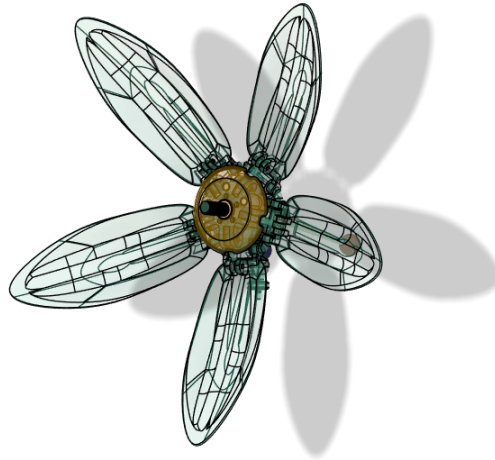


Figure 1: The fully assembled flower head.

1. Ensure all required parts are gathered and printed. See Appendix A.
2. Press the 3 brass sleeves into part BaseV3.
3. Glue the RedBall onto the end of one 3mmx50mm rod and the BlueBall onto the end of the other, then press the unglued end of each rod into BaseV3.
4. Glue ThreadedRod onto the spindle of the GearMotor, then use the M1.6 screws to affix the gear motor to BaseV3. The flower head should now look like Figure 2.

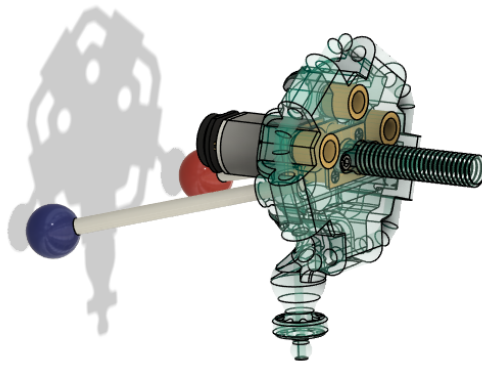


Figure 2: The flower head after the GearMotor and BaseV3 have been joined.

5. Use the pins to affix the smaller-diameter end of each connecting rod to a flower petal.
6. Press the M6x6 heat-set insert and the three 4x30mm rods into the CentreRingV3 structure piece. Ensure the rods sit flush against the side of CentreRingV3 that has the holes for the shell.
7. Use the pins to affix the larger-diameter end of each connecting rod (with its petal already attached) to the CentreRingV3 structure piece.
8. Press the CentreRingV3 shell piece onto the structure piece. The flower petal assembly should now match Figure 3.

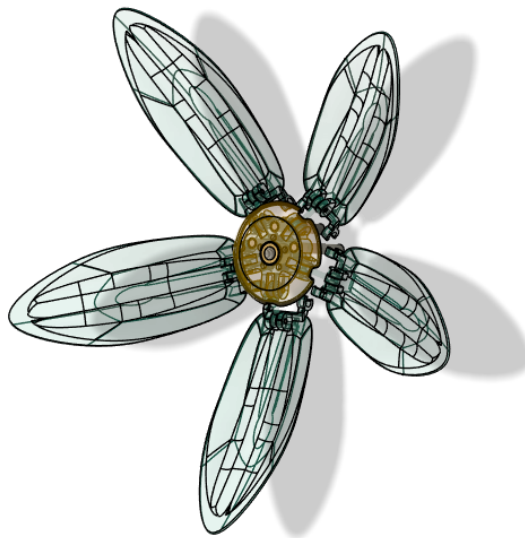


Figure 3: The flower petal and CentreRingV3 assembly, prior to attachment to BaseV3.

9. Line up the assembled CentreRingV3 with BaseV3. Spin the threaded rod, either by hand or by driving the motor, to draw the ring-and-petal assembly onto the base.
10. Pre-solder the WS2812 LEDs into a chain of 5 using the 3-conductor wire, then glue the chain onto the petals. The flower head is now fully assembled and should match Figure 1.

1.2 Stem



Figure 4: The fully assembled stem, showing the ball-and-socket joints threaded onto the spring steel rod.

1. Ensure all required parts are gathered and printed. See Appendix A.
2. Starting with the largest ball-and-socket joint at the bottom and working upward in decreasing size, thread each joint onto the 1.2 mm spring steel rod. Do not cut the rod to length yet; leave at least 10 cm of extra rod on each end. The final result should match Figure 4.

1.3 Flower Pot

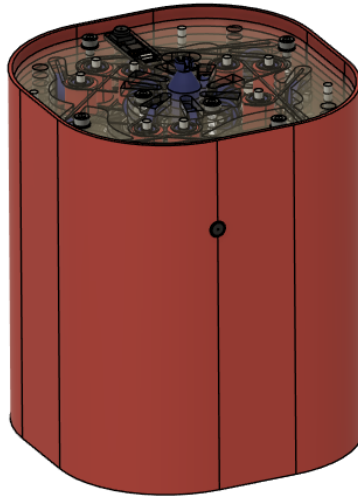


Figure 5: The fully assembled flower pot.

1.3.1 Servo Assembly

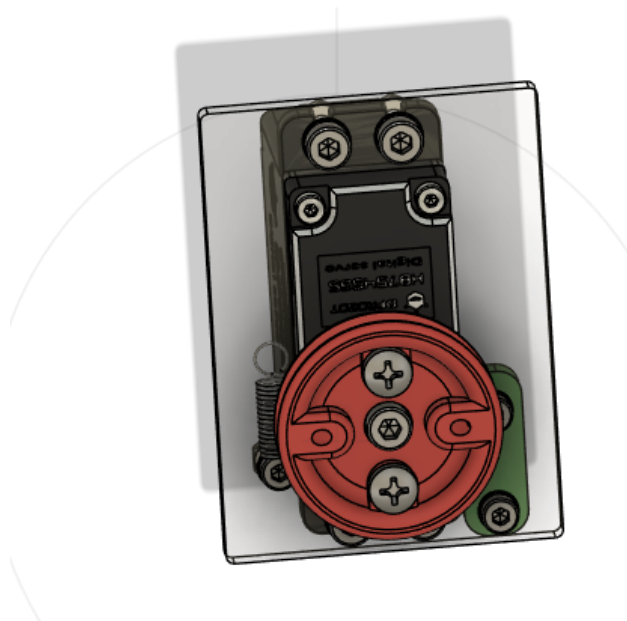


Figure 6: A completed servo assembly, showing the spool mounted with its wire holes horizontal.

Always add a nylon washer at every contact point between screw hardware and the laser-cut acrylic.

1. Ensure all required parts are gathered and printed. See Appendix A.

2. Using M3 hardware, affix the servo motors to the laser-cut plate.
3. Affix the spring using M2.5 hardware.
4. Press the M3 and M2.5 heat-set inserts into the ServoTensioners, then affix each ServoTensioner onto the acrylic plate, ensuring the side that received the M3 insert faces upward.
5. Using the M3 rounded Phillips screws, affix the spool to the servo horn, then screw the servo horn onto the servo. Best practice is to mount the spool onto the horn with the wire holes horizontal, matching the arrangement in Figure 6; to ensure this, zero the servos before attaching the horn.
6. Repeat this process to build all four servo assemblies.

1.3.2 Plates Assembly

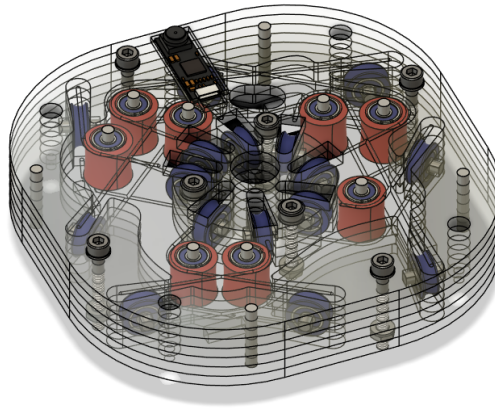


Figure 7: The completed Plates Assembly.

Always add a nylon washer at every contact point between screw hardware and the laser-cut acrylic.

1. Ensure all required parts are gathered and printed. See Appendix A.
2. Cover the underside of MidPlate with packing tape. Add a dab of super glue into each dowel hole, then press the alignment dowels and bearing dowels into place. Leave the 4 outer screw holes empty.
3. Apply a thin, even coat of super glue over the entire underside of MidPlateShell, then press it onto MidPlate. Hold under pressure until the glue cures.
4. Repeat the previous step with MidPlateShellBearing. The assembly should now look like Figure 8.

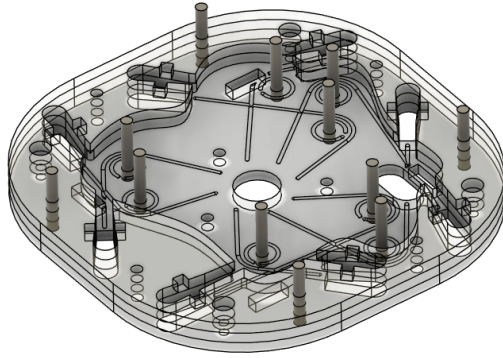


Figure 8: MidPlate after MidPlateShell and MidPlateShellBearing have been glued in place.

5. Add the remaining 2 MidPlateShells onto the alignment dowels.
6. Push a bearing into each side of a bearing sleeve and slide it onto the inner rods.
7. Push a bearing into a SmallBearingSleeve, slide it onto a 10 mm rod, and place it in the outer slots of MidPlateShellBearing.
8. Glue a second MidBearingMount-Bottom to a MidBearingMount-Mid, using screws or alignment dowels to keep the two pieces aligned.
9. Place the remaining bearings into the remaining SmallBearingSleeves, insert the 8 mm dowels, and place them into the glued MidBearingMount assembly from the previous step.
10. Place one MidBearingMount-Bottom in the middle of MidPlate, stack the glued bearing sub-assembly from the previous step on top of it, then add the final MidBearingMount-Bottom on top.
11. Fasten the middle-plate stack together with the 3 shorter M3 screws. The assembly should now mirror Figure 9.

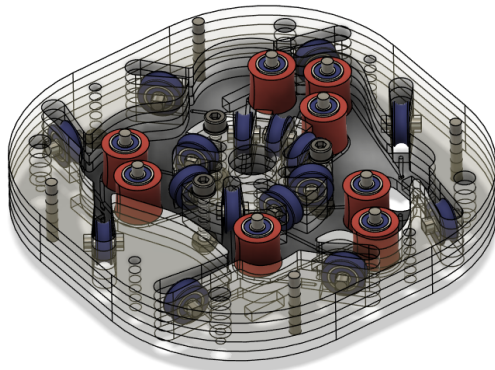


Figure 9: The middle-plate stack, bearings installed, fastened with the three shorter M3 screws.

12. Lastly, add TopPlate, affix it with the remaining 4 M3 screws, and press the pinhole camera into place. The assembly should mirror Figure 7.

1.3.3 Electronics Plate

Always add a nylon washer at every contact point between screw hardware and the laser-cut acrylic. Note that assembling the Perfboard sub-assembly itself is not covered in these instructions; it is assumed to already be built.

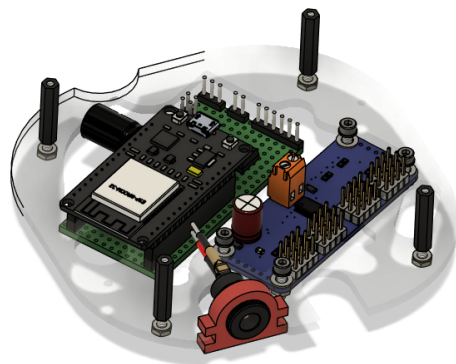


Figure 10: The completed Electronics Plate assembly.

1. Ensure all required parts are gathered and printed. See Appendix A.
2. Using M2.5 hardware, attach both the Perfboard (with the ESP32 mounted) and the servo motor driver.
3. Attach the barrel jack to the BarrelJackMount, then slide the assembly into position.
4. Affix the M2.5x24mm standoffs as spacers. This assembly should mirror Figure 10.

1.3.4 Assembling the Assemblies

Always add a nylon washer at every contact point between screw hardware and the laser-cut acrylic.

1. Ensure all required parts are gathered and printed. See Appendix A.
2. Add 3 M2.5x6mm and 1 M2.5x4mm heat-set inserts into each of the 4 Side Supports.
3. Affix all 4 Side Supports onto BasePlate using M2.5 hardware.
4. Slide the 4 servo assemblies into the side supports and screw the free end of each spring into its corresponding Side Support.
5. Insert the CentreSupport and ensure it is properly seated in BasePlate.
6. Add the 3 8mm standoffs under BasePlate for mounting the Electronics Plate. Reference Figure 11.

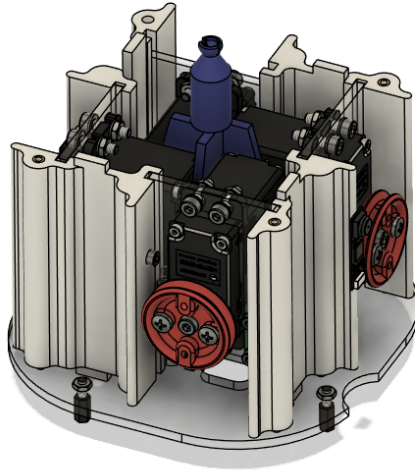


Figure 11: BasePlate with the Side Supports, servo assemblies, CentreSupport, and mounting standoffs installed.

7. Affix the Electronics Plate to the underside and the Plates Assembly on top.
8. Hot-glue the second pinhole camera into the hole in the outer shell, ensuring its cable points downward.
9. Slide the flower pot assembly into the outer shell. Ensure both pinhole camera cables exit through the power hole on the back of the outer shell and that the shell is oriented correctly. This completes the flower pot assembly; see Figure 5.

1.4 Assembling the Assemblies

1.4.1 Connecting the Flower Head with the Stem

Always add a nylon washer at every contact point between screw hardware and the laser-cut acrylic.

1. Ensure all required parts are gathered and printed. See Appendix A.
2. To connect the Flower Head to the Stem, press the top ball and socket into the receiving end of the Stem into the ball of the flower head. Ensure that the backbone enters the flower head and is almost fully extended through the head. See figure 12.

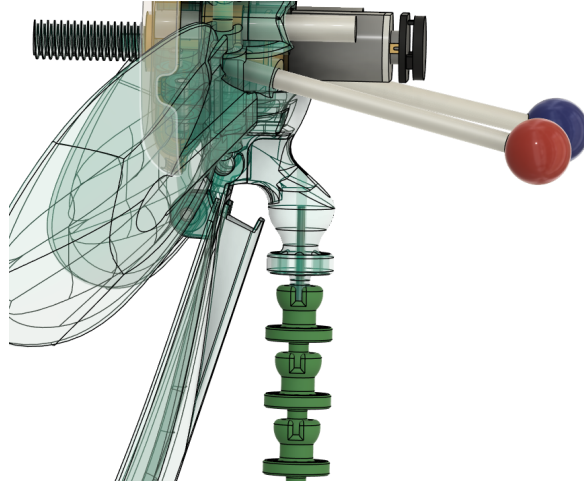


Figure 12: Connecting the Flower Head to the Stem

1.4.2 Stringing the Flower Pot and Final assembly

1. Remove the flower pot mechanics from the shell.
2. Unscrew the M2.5 screws affixing the plates assembly to the rest of the flower pot and remove the plates assembly.
3. Remove the TopPlate of the plates assembly.
4. Following the engraved paths in MidPlate, cut wire into approximately 50 - 75 cm lengths and thread through the middle bearings around the internal rollers, and lastly over the outer bearing through the bottom hole.
5. Wire both the Pinhole camera cable and 7 conductor cable used for flower head control through the associated base plate hole.
6. Wire the tendons through the top plate, ensuring that they go through their corresponding hole, then wire the 7 conductor cable and pinhole camera cable through their associated hole before re-affixing the top plate to the plates assembly.
7. Re-affix the plates assembly to the rest of the flower pot.
8. To wire the cables around the spools on the servo motor, wrap the cables around the spool one time entirely before threading through the holes on the inside surface of the spool. It can sometimes be difficult threading the wire through the small holes inside the spool, this is as the hole bends 90 degrees out, allowing the cables to be crimped. What can help with this process is bending the end of the cable to match this 90 degree curve.
9. IMPORTANT: Do not crimp the cables once wired through the spools. Crimping at the spools occurs after tendons are crimped at the flower head.
10. With the flower stem and head assembly not yet attached to the pot, thread the tendons through the ball and socket joints. For the lower stage, wire 2 of the tendon pairs through the first 6 ball and sockets, and for the upper stage, wire the remaining 2 pairs all the way up to the flower head.

11. Attach the flower stem to the flower pot. ensuring the the backbone slides into the flower pot.
12. IMPORTANT: Ensure all servo motors are zeroed before crimping and tendons.
13. Once the tendons are strung through both the flower stem and properly wired in the spools, first add a crimp to the stem side of the tendon and cut the cable flush with the crimp, then pull the same tendon from the spool until the crimp sits up against the ball and socket piece. Next pull tension on the cable exiting the spool and affix the second crimp as close to the spool as possible. Cut access cable and repeat for all 8 tendons.
14. Using the Tensioning screws on the flower pot, adequately tension each tendon set. For more information see section 2.1.
15. Connect all wiring in the Electronics plate.
16. Slide the Flower pot Outer Shell back over the mechanics.
17. Add lubricant to holes which the tendons run through on the ball and socket joints
18. Connect the wiring connectors for the LED lights and N20 GearMotor.
19. Test to ensure that the upper and lower stage control in the code is assigned to the proper servo motors after stringing the flower.

2 Repairs & Tuning

2.1 Tensioning the Tendons

To adjust tension in the tendons:

1. Lift the flower pot assembly out of the case.
2. With a 2.5 mm Allen key, turn the M3 bolt associated with the tendon pair being adjusted (clockwise to tighten, counter-clockwise to loosen). The screws can be seen in figure 13.

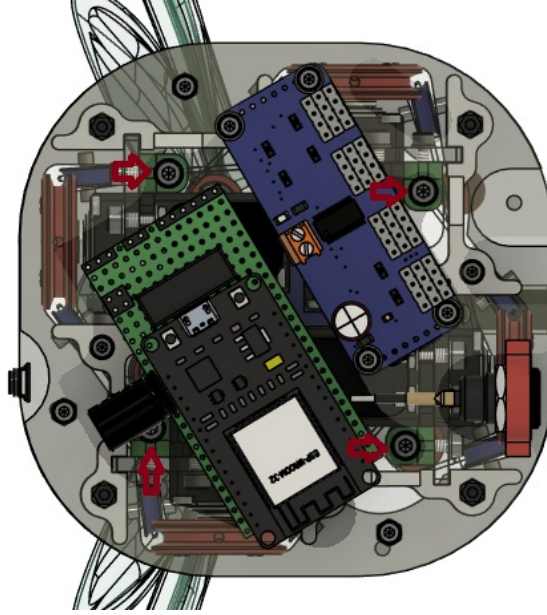


Figure 13: Bottom of Flower Pot showing the Tensioning screws

3. The tendons are tight enough once there is less than approximately 5mm of backlash when pressing with your hand on the tendons between the plates assembly and the spool.
4. Lower the flower pot assembly back into the case.

2.2 Re-stringing Tendons

Re-Stringing the Tendons may be needed either for a repair or if there is no longer enough space to further tension the cables. It is recommended that both tendons associated with a given servo motor be restrung at the same time.

1. Lift the flower pot assembly out of the case.
2. With a 2.5 mm Allen key, fully loosen the M3 bolt associated with the tendon pair being replaced.
3. Cut the tendon as close to the crimp on the flower stem.
4. Pull the cut tendon from just under the very first ball and socket joint. If done correctly the tendon should be sticking out from the flower pot plates assembly without going through any of the ball and sockets.
5. Cut the new length of cable (Approx 50 - 75 cm) and apply super glue to glue the cable sticking out from the plates assembly to the end of the new cable.
6. Cut the cable on the spool end and pull the tendon through the plates assembly, pulling the new cable glued on with it. Once the new cable comes through the other end, keep pulling until the cable almost fully enters the plates assembly on the ball and socket side.
7. Using fine needle nose pliers or tweezers, pull the new cable up through the ball and socket joints

8. Crimp the new cable above the ball and socket joint and pull tension on the spool side.
9. To wire the cables around the spools on the servo motor, wrap the cables around the spool one time entirely before threading through the holes on the inside surface of the spool. It can sometimes be difficult threading the wire through the small holes inside the spool, this is as the hole bends 90 degrees out, allowing the cables to be crimped. What can help with this process is bending the end of the cable to match this 90 degree curve.
10. Once the tendons are strung through both the flower stem and properly wired in the spools, pull tension on the cable exiting the spool and affix the spool crimp as close to the spool as possible.
11. Using the Tensioning screws on the flower pot, adequately tension each tendon set. For more information see section 2.1.
12. Slide the Flower pot Outer Shell back over the mechanics.
13. Add lubricant to holes which the tendons run through on the ball and socket joints.

3 Software Overview

This section explains how the flower’s software is put together. Section 4 covers laptop dependencies, and Section 5 covers startup and troubleshooting.

3.1 Communication

The flower communicates over WiFi between a host and Microcontroller (ESP-32). The two systems communicate over Wi-Fi, using ROS 2 and micro-ROS.

- **The ESP32:** It runs the embedded firmware; C++ that directly drives the four stem servos, the five petal LEDs, and the petal angles.
- **Host Machine:** A host running Ubuntu 22.04 with ROS 2 installed (see Section 4 for full requirements). It runs; the GUI, the logic that decides which commands actually reach the hardware, a forward kinematics solver, and two camera-based perception nodes.

3.2 Architecture at a Glance

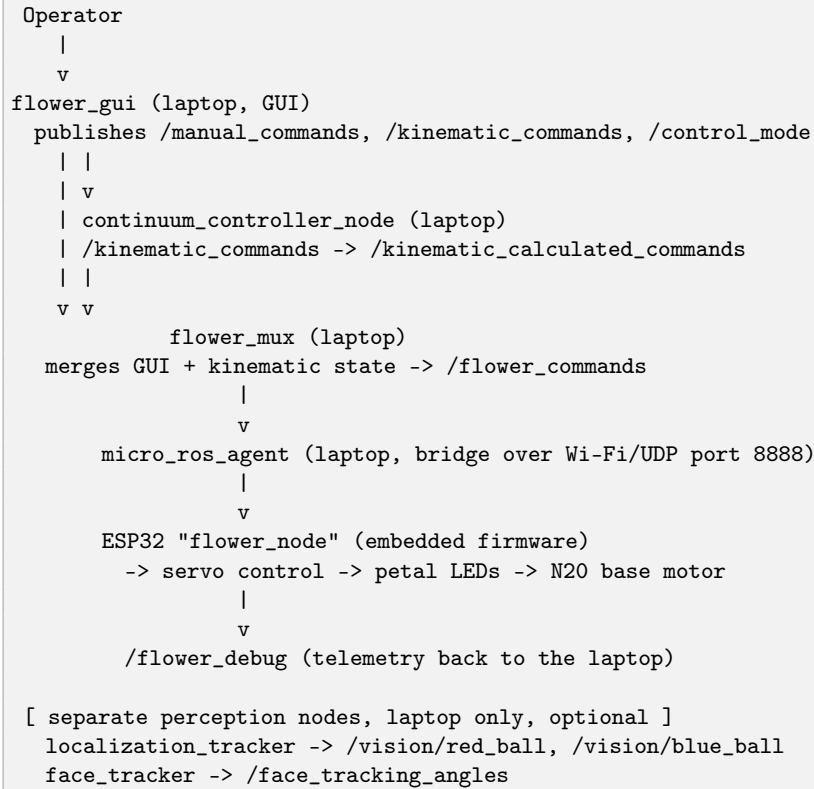


Figure 14: Simplified software/data flow. Everything above the Wi-Fi bridge runs on the laptop; everything below it runs on the ESP32.

3.3 ROS 2 Topics Reference

Table 1: ROS 2 topics used by the flower software stack.

Topic	Published by	Purpose
/manual_commands	flower_gui	Full manual command state from the GUI: 4 servo angles + move times, LED colours + brightness (x5), and N20 motor target/speed/zero flag. Published continuously at 10 Hz.
/kinematic_commands	flower_gui	[theta1, phi1, theta2, phi2] in degrees — the bend magnitude and bend-plane direction requested for each of the stem's two continuum stages.
/control_mode	flower_gui	The string "manual" or "kinematic", selecting which command source flower_mux should forward.

Table 1 – continued from previous page

Topic	Published by	Purpose
/kinematic_calculated_commands	continuum_controller_node	The four per-servo angles solved from the requested bend targets (each stage’s bend is realized as a pair of antagonistic servo angles).
/flower_commands	flower_mux	The single, final, merged command — this is the only topic the ESP32 actually receives, via the micro-ROS agent.
/flower_debug	ESP32 (flower_node)	Plain-text telemetry published by the firmware: relayed copies of received commands (for confirming round-trip delivery) and status messages. Useful for troubleshooting.
/vision/red_ball, /vision/blue_ball	localization_tracker	3D position of the red/blue localization balls (Section A, Flower Head parts) as seen by a USB camera. Not currently consumed by any other node; reserved for future closed-loop control.
/face_tracking_angles	face_tracker	Pan/tilt angular offset (degrees) of the closest detected face from a second USB camera, for future attention-based behaviour.

3.4 Repository Layout

Table 2: Top-level folders in the project repository.

Folder	Contents
FullCode_ws/	The ROS 2 host workspace: the GUI, command mux, kinematics controller, message definitions, perception nodes, the micro-ROS agent source, and the top-level <code>flower.launch.py</code> that starts everything together.
FullCodeMICROCONTROLLER/	The embedded firmware project (PlatformIO), compiled and uploaded to the ESP32.
Testing/	Standalone hardware/software experiments and validation sketches (Wi-Fi tests, servo tests, kinematic prototyping, etc.) — not part of the deployed system.
Construction/	CAD exports, laser-cut sketches, and the Bill of Materials (Appendix A; source files indexed in Appendix B).

3.5 Embedded Firmware (ESP32)

The firmware lives in `FullCodeMICROCONTROLLER/` and is built with PlatformIO. Every loop iteration it process any incoming network message, then push the latest commanded state out to the servos, LEDs, and motor.

Table 3: Embedded firmware source files and responsibilities.

File	Responsibility
<code>src/main.cpp</code>	Entry point. Runs startup (LEDs, Wi-Fi/micro-ROS, servos, N20 motor, in that order) then loops forever: pumps ROS callbacks, updates LEDs/servos at ~ 33 Hz, drives the N20 control loop, and watches the agent connection so it can reconnect automatically if it drops.
<code>src/MicroRos.cpp</code>	The Wi-Fi and micro-ROS bridge: connects to Wi-Fi, resolves the laptop’s hostname over mDNS, pings the agent, subscribes to <code>/flower_commands</code> , and publishes <code>/flower_debug</code> . This is the file described in detail in Section 3.7.
<code>src/servo_control.cpp</code>	Drives the 4 stem servos through a PCA9685 I2C servo driver, smoothly easing each one toward its commanded angle over a commanded time window.
<code>src/petal_lights.cpp</code>	Drives the 5 WS2812B petal LEDs (via the FastLED library). Also implements the connection-status “spin” animation described in Section 5.4.
<code>src/N20.cpp</code>	Controls the base’s N20 geared motor and reads its quadrature encoder to estimate and hold a target position.
<code>include/MicroRosWifiCredentialsEXAMPLE.h</code>	Template for the Wi-Fi/agent credentials file. Copied to <code>MicroRos</code>
<code>platformio.ini</code>	<code>WifiCredentials.h</code> and filled in locally — see Section 4.4.
<code>extra_packages/flower_msgs/</code>	Build configuration: target board (<code>esp32dev</code>), ROS distro (<code>humble</code>), transport (<code>wifi</code>), and library dependencies (FastLED, Adafruit PWM Servo Driver Library, <code>micro_ros_platformio</code>). Also sets <code>board_microros_extra_packages=extra_packages</code> , pointing at the vendored message package below.
	Copy of the <code>flower_msgs</code> package (same <code>RobotCommand.msg</code> fields as the host-side copy in <code>FullCode_ws</code>). The micro-ROS PlatformIO build compiles its own micro-ROS library from source and needs every custom message type available locally to do it, so this copy has to be updated by hand whenever <code>RobotCommand.msg</code> changes — see the PIO clean/rebuild note in Section 4.4.

3.6 Host Software Stack

Everything in this subsection runs on the host, inside the `FullCode_ws` ROS 2 workspace, and is started with `flower.launch.py` (Section 5).

- **flower_gui**: A Tkinter desktop window running alongside a ROS 2 node. It has a *manual* mode (direct sliders for each of the 4 servos) and a *kinematic* mode (theta/phi bend sliders for each of the stem's two stages, with a live 3D preview of the resulting shape). Each of the 5 LEDs, and the N20 motor controlling the petal angle are controlled through the GUI. All Servo sliders have a time that can be inserted next to them: this is used to adjust the total movement time i.e. how fast/slow a movement should be. **Safety note:** The manual servo sliders should be used with caution over 45 DEGREES large commanded angles can over-stress the ball-and-socket joints. Furthermore, before any fast actions are performed, it is good practice to mimic the movement slowly to ensure the goal position is within capabilities of the robot. GUI settings (last-used mode, slider values, etc.) are saved automatically to `~/flower_gui_settings.json`, so the dashboard reopens in its last configuration.
- **flower_mux**: Watches `/control_mode` and forwards either the raw manual state or the kinematic-solver output to `/flower_commands`, while making sure the LED and N20 fields from the GUI are always preserved regardless of mode.
- **controller** (`continuum_controller_node`): Kinematics solver. Converts each stage's requested bend (`theta`, `phi`) into the pair of antagonistic servo angles that physically produce that bend. Potential future work could be preformed here to add localization to control.
- **flower_msgs**: Defines the custom `RobotCommand` message type shared by the GUI, mux, controller, and firmware.
- **localization_tracker** (camera index 0) and **face_tracker**: (camera index 2) Perception nodes, each expecting its own dedicated USB camera at that hardcoded `cv2.VideoCapture` index. The former uses HSV colour segmentation (with circularity/solidity shape-filtering to reject glare) to find the red/blue localization balls (Section A) and EMA-smooths the published position; it also locks the camera's brightness/contrast/white-balance/exposure via `v4l2-ctl` on startup so lighting auto-adjustments don't drift the colour thresholds mid-session. If the camera is replaced, its intrinsics need recalibrating with:

```
ros2 run usb_cam usb_cam_node_exe
ros2 run camera_calibration cameracalibrator --size 9x6 --square 0.020 --ros-args
-r image:=/image_raw
```

The latter uses RetinaFace to find and publish the nearest face's pan/tilt angle. **Neither node's output is currently hooked up to anything.**

- **micro_ros_agent**: Wi-Fi/UDP-to-ROS bridge.
- **foxglove_bridge**: Exposes all ROS 2 topics to [Foxglove Studio](#) for debugging.

3.7 Wi-Fi / micro-ROS Connection Protocol

On every boot (and on every reconnect after a dropped connection), the firmware in `MicroRos.cpp` performs these steps in order to reconnect to the host machine:

1. **Connect to Wi-Fi.** The firmware attempts `WIFI_SSID_1` for up to ~ 10 seconds. If that fails, it switches over (SSID, password, *and* target agent hostname) to `WIFI_SSID_2` and tries again for another ~ 10 seconds. This lets one flower unit be pre-configured for two different networks without reflashing. While this step is in progress, a single yellow LED chases around the 5 petals.
2. **Resolve the laptop's address.** Once on Wi-Fi, the ESP32 starts its own mDNS responder (as `flower-node.local`) and then queries mDNS for whichever `AGENT_HOSTNAME` corresponds to the network it joined, to get the laptop's current IP address.
3. **Ping the micro-ROS agent.** Once the laptop's IP is known, the firmware configures its network transport to talk to that IP on `AGENT_PORT` (8888 by default, UDP) and repeatedly pings the agent for up to ~ 5 seconds. While resolving the hostname and pinging, a single green LED chases around the 5 petals.
4. **Bring up ROS entities.** Once the agent responds, the firmware creates its ROS node, subscribes to `/flower_commands`, and creates the `/flower_debug` publisher. The petal LEDs are cleared (all off) once this finishes successfully, and normal operation begins.

The main loop also pings the agent every 2 seconds during normal operation; if a ping fails, the firmware tears down and automatically retries this entire sequence.

4 Host Setup

This section explains the process that needs to be installed and built *once* per laptop. Once done, refer to Section 5 for day-to-day operation.

4.1 Supported Configuration

The host stack is built and tested against **Ubuntu 22.04 LTS** running **ROS 2 Humble**. Other configurations may work but are not documented.

4.2 Required Software & Dependencies

Table 4: Software that must be installed on the laptop before the flower can be operated.

Component	Purpose	Required for
ROS 2 Humble (<code>ros-humble-desktop</code>)	Core messaging middleware; provides <code>ros2</code> , <code>rclpy</code> , and the standard message types.	Everything
<code>python3-colcon-common-extensions</code>	Build tool for the ROS 2 workspace.	Building <code>FullCode_ws</code>
<code>python3-rosdep</code>	Resolves and installs each package's declared dependencies automatically.	Building <code>FullCode_ws</code>

Table 4 – continued from previous page

Component	Purpose	Required for
ros-humble-foxglove-b ridge	Exposes ROS topics to Foxglove Studio for debugging.	<code>flower.launch.py</code>
git	Cloning the repository and the micro-ROS agent sources.	Everything
python3-numpy, python3-matplotlib, python3-tk	Numerical/plotting/GUI-toolkit support for the kinematics preview.	<code>flower_gui</code>
python3-opencv (or pip install opencv-python), ros-hu mble-cv-bridge	Camera capture and image handling.	<code>localization_tracker</code> , <code>face_tracker</code>
pip install retina-face	Face detection model.	<code>face_tracker</code> only
PlatformIO Core (pip install platformio) or the PlatformIO IDE extension for VS Code	Compiles and uploads the embedded firmware.	Flashing the ESP32 only

The `face_tracker` node’s optional GPU acceleration expects CUDA libraries under `~/local/lib/python3.10/site-packages/nvidia/` (i.e. installed via pip’s `nvidia-*` packages, as pulled in by a CUDA-enabled PyTorch install). `flower.launch.py` adds these to `LD_LIBRARY_PATH` automatically, so no manual environment configuration is needed once the packages are installed.

4.3 Cloning & Building the ROS 2 Workspace

```
# 1. Clone the repository
git clone <repository-url>
cd llm4mentalhealth/FullCode_ws

# 2. Populate the vendored micro-ROS packages (src/micro_ros_setup,
# src/uros/micro-ROS-Agent, src/uros/micro_ros_msgs). These folders are
# tracked as git submodule links but the .gitmodules registration is not
# committed to this repository, so a fresh clone leaves them EMPTY. If
# 'ls src/micro_ros_setup' shows nothing, populate them manually:
git clone -b humble https://github.com/micro-ROS/micro_ros_setup.git src/
micro_ros_setup
git clone -b humble https://github.com/micro-ROS/micro-ROS-Agent.git src/uros/micro-ROS
-Agent
git clone -b humble https://github.com/micro-ROS/micro_ros_msgs.git src/uros/
micro_ros_msgs

# 3. Source the base ROS 2 install (needed in every new terminal)
source /opt/ros/humble/setup.bash

# 4. Resolve and install every package’s declared dependencies
rosdep update
```

```

rosdep install --from-paths src --ignore-src -r -y

# 5. Build the whole workspace
colcon build

# 6. Source the newly built workspace (also needed in every new terminal,
# AFTER sourcing the base ROS 2 install above)
source install/setup.bash

```

A successful build creates `build/`, `install/`, and `log/` folders (all git-ignored). If `colcon build` fails on a specific package, re-run `rosdep install`

4.4 Flashing the ESP32 Firmware

This step only needs to be repeated when the ESP32 firmware changes, or when the flower needs to be configured for a different Wi-Fi network (Section 5.3). It does *not* need to be repeated every time the flower is used.

1. Remove the ESP32 from the proto board.
2. Connect the ESP32 to the laptop with a USB cable.
3. If this is the first time flashing, create the credentials file from the template:

```

cd FullCodeMICROCONTROLLER
cp include/MicroRosWifiCredentialsEXAMPLE.h include/MicroRosWifiCredentials.h

```

4. Edit `include/MicroRosWifiCredentials.h` and fill in real values for all seven fields:

```

#define WIFI_SSID_1 "YourNetworkName"
#define WIFI_PASS_1 "YourNetworkPassword"
#define AGENT_HOSTNAME_1 "your-laptop-hostname" // laptop's mDNS hostname on that
network

#define WIFI_SSID_2 "BackupNetworkName" // optional second network
#define WIFI_PASS_2 "BackupNetworkPassword"
#define AGENT_HOSTNAME_2 "your-laptop-hostname" // usually the same laptop/
hostname

#define AGENT_PORT 8888 // must match the port micro_ros_agent listens on

```

`AGENT_HOSTNAME_1/2` must exactly match the laptop's own hostname. If only one network is available.

5. Build and upload the firmware. **IMPORTANT: If the microROS Flower Messages File is edited, a complete PIO Clean and rebuild is REQUIRED.**

5 Startup Instructions

This section assumes the one-time laptop setup in Section 4 is already complete (ROS 2 and dependencies installed, the workspace built, and the firmware already flashed with valid Wi-Fi credentials). If that has not been done yet, complete Section 4 first.

5.1 Every-Day Startup

1. Power on the flower by plugging in the barrel jack (Section 1.3.3). The ESP32 boots immediately and begins the Wi-Fi connection sequence described in Section 3.7: the petal LEDs will start chasing yellow. **Ensure that the flower is powered on before host code is ran.**
2. Ensure the laptop is connected to the same Wi-Fi network configured in the flower's credentials file (WIFI_SSID_1 or WIFI_SSID_2).
3. Open a terminal on the laptop and source the environment:

```
source /opt/ros/humble/setup.bash
source ~/llm4mentalhealth/FullCode_ws/install/setup.bash
```

4. Launch the full host stack (agent, GUI, mux, kinematics controller, perception nodes, and the Foxglove bridge) with a single command, run from inside FullCode_ws:

```
cd ~/llm4mentalhealth/FullCode_ws
ros2 launch flower.launch.py
```

5. Watch the flower's petal LEDs: they should go from spinning yellow, to spinning green, to connected (see Section 5.4).
6. Use the GUI (Section 3.6) to drive the flower manually or in kinematic mode.

5.2 First-Time Startup on a New Flower Unit or New Wifi

1. Complete the one-time laptop setup in Section 4 if it has not already been done on this laptop.
2. Flash the firmware with Wi-Fi credentials for the network, following Section 4.4.
3. Follow the Every-Day Startup steps above.

5.3 Connecting to a Different Wi-Fi Network

The flower does not scan for networks or accept credentials at runtime. To move it to a new network:

1. On the host laptop on the new network, confirm its hostname with `hostname` and confirm it is discoverable over mDNS.
2. Edit `FullCodeMICROCONTROLLER/include/MicroRosWifiCredentials.h` and update `WIFI_SSID_1/WIFI_PASS_1/AGENT_HOSTNAME_1` (or the `_2` fields).
3. Reflash following Section 4.4.

Remember: the firmware tries Network 1 for ~10 seconds, and only falls back to Network 2 if Network 1 is unreachable, so if both fields point at networks that are simultaneously in range, Network 1 always wins.

5.4 LED Status Indicator & Troubleshooting

The five petal LEDs double as a connection-status indicator during startup and reconnection.

Table 5: Petal LED patterns and what they mean.

Pattern	Meaning	If it doesn't move on
One yellow LED chasing around the 5 petals	Trying to join Wi-Fi (Network 1, then Network 2 as a fallback).	Double-check <code>WIFI_SSID_1/WIFI_PASS_1</code> (and <code>_2</code>) in the credentials file; confirm the flower is powered and within range of that router; confirm the router is actually up. Each network is retried for ~ 10 seconds before falling back.
One green LED chasing around the 5 petals	Wi-Fi connected; now resolving the laptop's mDNS hostname and pinging the micro-ROS agent.	Confirm the laptop is powered on, joined to the <i>same</i> Wi-Fi network, and running <code>ros2 launch flower.launch.py</code> (which starts the agent); confirm <code>AGENT_HOSTNAME_1/2</code> exactly matches the laptop's real hostname; confirm no firewall is blocking UDP port 8888 or mDNS (UDP 5353).
Petals showing steady, non-spinning colours matching the GUI sliders	Normal steady-state operation. The ESP32 is connected and actively receiving commands.	Nothing — this is the expected state during use.
Petals resume spinning yellow/green after previously being steady	The connection to the agent was lost (a ping over UDP failed) and the firmware is automatically retrying the full reconnect sequence.	Check that the <code>ros2 launch flower.launch.py</code> terminal is still running on the laptop, that the laptop hasn't disconnected from Wi-Fi or gone to sleep, and that the flower is still in range.
No LED activity at all , ever, after power-on	No power reaching the board, the firmware crashed before LED init, or the LED data wire is disconnected/damaged.	Check the barrel jack and internal power wiring; check the LED signal wire soldered during Flower Head assembly (Section 1.1); connect over USB and run <code>pio device monitor</code> (Section 4.4) to read the boot log for a crash message.

5.5 Shutting Down

1. Return all servos to a neutral/resting position via the GUI, so the flower does not sit under excessive tension.

2. Press `Ctrl+C` in the `ros2 launch flower.launch.py` terminal to stop every host-side node cleanly.
3. Unplug the flower's barrel jack.

5.6 Quick Command Reference

Table 6: Frequently used commands, for reference.

Purpose	Command
Source ROS 2 (every new terminal)	<code>source /opt/ros/humble/setup.bash</code>
Source the workspace (every new terminal, after the line above)	<code>source install/setup.bash</code>
Build/rebuild the host workspace	<code>colcon build</code> (run from <code>FullCode_ws</code>)
Start the full host stack	<code>ros2 launch flower.launch.py</code> (run from <code>FullCode_ws</code>)
Watch firmware telemetry	<code>ros2 topic echo /flower_debug</code>
List all active topics	<code>ros2 topic list</code>
Watch the ESP32's serial boot log	<code>pio device monitor</code>

Appendix

A Bill of Materials

This appendix lists the complete Bill of Materials (BOM) for the FlowerAssembly, as exported on August 18, 2026. Table 7 breaks the parts down by sub-assembly/section, and Table 8 gives the consolidated (deduplicated) part counts across the whole assembly, with source links where available.

Table 7: FlowerAssembly Bill of Materials — individual counts by sub-assembly.

Part Name	Description	Qty	Material
Flower Head			
BaseV3	Central Component	1	Resin (MSLA Printer)
CenterRingV3	2 Bodies, Structure & Shell. Print separately. Holes drilled to 2mm for Pins	1	Resin (MSLA Printer)
ThreadedRod	Lead screw for gearmotor	1	Resin (MSLA Printer)

Continued on next page

Table 7 – continued from previous page

Part Name	Description	Qty	Material
BlueBall	For Localization	1	PLA (FDM Printer)
3mmx50mm rod	Cut Stainless Steel Rod, Supports Balls	2	Steel
FlowerPetal	Flower Petals, Holes drilled to 2mm	5	Resin (MSLA Printer)
RedBall	For Localization	1	PLA (FDM Printer)
4x30mm rod	Cut Stainless Steel Rod, Centering Rods for CentreRingV3	3	Steel
ConnectingBracketV3	Connecting Rods Between Flower Petal and Center Ring. Holes Drilled to 2mm for Pins	5	Resin (MSLA Printer)
4x6mm Brass Sleeve	Sleeve for reduced friction on centering rods	3	Brass
M6x6 Heat Set Insert	Placed in CenterRingV3	1	Brass
Socket Head Cap Screw M1.6x0.35 x 4	N20 Motor Screws	2	Steel
MicroGearmotor v3	300RPM 6V N20 GearMotor	1	Brass, Soft Yellow
ws2812 5050 led	LED's for the Flower Petals	5	Electrical Component
3 wire conductor	3 wire conductor for LED's in ft	1	Electrical Component
JST1.5 6 pin connector male	Connector for N20 Motor	1	Electrical Component
JST1.5 3 pin connector	Male and Female for connector for LED string	1	Electrical Component
Pin	Cut 2x8mm Stainless Steel Rod. Attaches Connecting Rod to both Petals and CenterRing	10	Steel
Stem			
Ball and Socket	Middle Joints	6	Resin (MSLA Printer)
Ball and Socket Top	Top Piece	1	Resin (MSLA Printer)
Ball and Socket - 4mm	Lower Joints Gradually Grow to connection with Base	1	Resin (MSLA Printer)
Ball and Socket - 7mm	Lower Joints Gradually Grow to connection with Base	1	Resin (MSLA Printer)

Continued on next page

Table 7 – continued from previous page

Part Name	Description	Qty	Material
Ball and Socket - 8mm	Lower Joints Gradually Grow to connection with Base	1	Resin (MSLA Printer)
Ball and Socket - 5mm	Lower Joints Gradually Grow to connection with Base	1	Resin (MSLA Printer)
Ball and Socket - 6mm	Lower Joints Gradually Grow to connection with Base	1	Resin (MSLA Printer)
1.2mm Spring Steel Rod	Back Bone	1	Spring Steel
Stainless Steel Cable, 1x7, 0.021"	Tendons. Each approx 1.5ft.	8	Steel
Aluminum Cable Ferrules, 0.02"	Crimps to tie Tendon Ends	16	Aluminum
Base			
<i>Servo Assembly (x_4)</i>			
ServoPlate	Plates that Servos Mount on	1	3mm Acrylic
Nylon Washer - M2.5	Nylon Washers For Spring and ServoTensioner Affixment	4	Nylon
Nylon Washer - M3	Nylon Washers for Servo Affixment	4	Nylon
Stainless Steel Washer - M3	Steel Washers for Servo Affixment	4	Steel
ServoTensioner	Component that allows for the assembly to be tensioned	1	PLA (FDM Printer)
HeatSetInsert-M2.5x2.5	To affix ServoPlate to ServoTensioner	2	Brass
SpoolV2	Wire Spool	1	Resin (MSLA Printer)
ServoHorn	Metal Circular Servo Horn Included with Servos	1	Steel
Spring-4mmx10	Spring for Tensioning	1	Steel
ServoMotorBrushless	Servo Motor	1	Electrical Component
Socket Head Screw - M3x0.5 x 12	Screws to Fasten Servo Motor	4	Steel
Hexagon Nut - M3 x 0.5	Nuts to Fasten Servo Motor	4	Steel
HeatSetInsert-M3x8	Heat Set Insert for Servo Tensioner	1	Brass
Socket Head Screw - M2.5x0.45 x 6	Screws to Fasten Servo Tensioner and Spring	3	Steel
Hexagon Nut - M2.5 x 0.45	Nut to affix Spring	1	Steel
Binding Head Screw - M3x5	Screws to affix SpoolV2 to Servo Horn	2	Steel
Socket Head Screw - M3x0.5 x 5	Screw to affix ServoHorn to ServoMotor	1	Steel

Continued on next page

Table 7 – continued from previous page

Part Name	Description	Qty	Material
<i>Side Supports (x4) - 1 Modified</i>			
HeatSetInsert-M2.5x6	Inserts for Mounting	3	Brass
HeatSetInsert-M2.5x4	Insert for Spring	1	Brass
SideSupports	Support to hold Servo Assemblies	1	PLA (FDM Printer)
SideSupportWire	Support to hold Servo Assemblies → 1x needed for wire passthrough	1	PLA (FDM Printer)
Socket Head Screw M2.5x0.45 x 4	Screw to affix Spring	1	Steel
<i>Plates Assembly (x1)</i>			
MidBearingMount-Mid	Plate that has bearing pin slots for middle section	1	3mm Acrylic
TopPlate	Top Plate	1	3mm Acrylic
3x18mm Rod	Holds assembly Together and supports Bearing	12	Steel
3x8mm Rod	Pins for inner bearings	8	Steel
Bearing	Bearings	24	Steel
SmallBearingSleeve	Vertical Bearing Sleeves	16	Steel
MidBearingMount-Bottom	Plate for middle sections	3	3mm Acrylic
MidPlate	Lower Assembly Plate	1	3mm Acrylic
3x12mm Rod	Pins for OuterBearings	8	Steel
MidPlateShell	Outer Acrylic	3	3mm Acrylic
MidPlateShellBearing	Outer Acrylic with bearing shaft holes	1	3mm Acrylic
Rollers	Horizontal bearing sleeves	8	PLA (FDM Printer)
Socket Head Screw M3x0.5 x 25	Outer Assembly Screws	4	Steel
Nylon Washer M3	Nylon Washers	14	Nylon
Nut M3 x 0.5	Nuts	7	Steel
Pinhole Camera	Ultra Tiny USB Webcam Camera with GC0307 Sensor	1	Electrical Component
Socket Head Screw M3x0.5 x 20	Inner Mid Plate Screws	3	Steel
<i>Electronics Assembly</i>			
Perfboard	Mounts Motor Driver and ESP32 and gives pinouts/ins for components	1	Perfboard

Continued on next page

Table 7 – continued from previous page

Part Name	Description	Qty	Material
1000uF Cap	Large Cap next to power in of Perfboard	1	Electrical Component
0.1uF Cap	Small Cap next to ESP32 Power Pins	1	Electrical Component
ESP-32	Microcontroller	1	Electrical Component
L293D H bridge Motor Driver	Motor Driver for N20 Motor	1	Electrical Component
ElectronicsPlate	Laser Cut plate that Electronics mount to	1	3mm Acrylic
PCA9685	Servo Motor Controller	1	Electrical Component
BarrelJackMount	Part to hold the barrel Jack	1	PLA (FDM Printer)
Barrel Jack	Power Jack	1	Electrical Component
Standoff m2.5x24mm	Flower Pot Legs	4	Plastic
Standoff m2.5x8mm	Standoff between base plate and electronics plate	1	3mm Acrylic
M2.5 Nut	Misc nuts	13	Steel
M2.5 Nylon Washer	Misc Washers	13	Nylon
M.25x8 Socket Head Screw	misc screws	7	Steel
<i>MISC</i>			
Nylon Washer M2.5	Misc Nylong Washers	22	Nylon
Socket Head Screw - M2.5x0.45 x 8	Misc Screws	12	Steel
CentreSupport	3D Printed Center Support	1	PLA (FDM Printer)
Nylon Washer M3	Washers for tensioning Screws	4	Nylon
7 Conductor Cable	Wiring between flower head and pot (ft)	2	Electrical Component
OuterShell	Case For Flower Pot	1	PLA (FDM Printer)
BasePlate	Laser cut base plate	1	3mm Acrylic
Socket Head Screw - M3x0.5 x 30	Tensioning Screws	4	Steel
Nut M2.5	Nuts	7	Steel
Pinhole Camera	Ultra Tiny USB Webcam Camera with GC0307 Sensor	1	Electrical Component

Table 8: FlowerAssembly Bill of Materials — consolidated total counts.

Part Name	Description	Qty	Material	Source
0.1uF Cap	Small Cap next to ESP32 Power Pins	1	Electrical Component	—
1.2mm Spring Steel Rod	Back Bone	1	Spring Steel	Link
1000uF Cap	Large Cap next to power in of Perfboard	1	Electrical Component	—
3 wire conductor	3 wire conductor for LED's in ft	1	Electrical Component	Link
3x12mm Rod	Pins for OuterBearings	8	Steel	Link
3x18mm Rod	Holds assembly Together and supports Bearing	12	Steel	—
3x50mm rod	Cut Stainless Steel Rod, Supports Balls	2	Steel	—
3x8mm Rod	Pins for inner bearings	8	Steel	—
4x30mm rod	Cut Stainless Steel Rod, Centering Rods for CentreRingV3	3	Steel	Link
4x6mm Brass Sleeve	Sleeve for reduced friction on centering rods	3	Brass	Link
7 Conductor Cable	Wiring between flower head and pot (ft)	2	Electrical Component	Link
Aluminum Cable Ferrules, 0.02"	Crimps to tie Tendon Ends	16	Aluminum	Link
Ball and Socket	Middle Joints	6	Resin (MSLA Printer)	—
Ball and Socket - 4mm	Lower Joints Gradually Grow to connection with Base	1	Resin (MSLA Printer)	—
Ball and Socket - 5mm	Lower Joints Gradually Grow to connection with Base	1	Resin (MSLA Printer)	—

Continued on next page

Table 8 – continued from previous page

Part Name	Description	Qty	Material	Source
Ball and Socket - 6mm	Lower Joints Gradually Grow to connection with Base	1	Resin (MSLA Printer)	—
Ball and Socket - 7mm	Lower Joints Gradually Grow to connection with Base	1	Resin (MSLA Printer)	—
Ball and Socket - 8mm	Lower Joints Gradually Grow to connection with Base	1	Resin (MSLA Printer)	—
Ball and Socket Top	Top Piece	1	Resin (MSLA Printer)	—
Barrel Jack	Power Jack	1	Electrical Component	—
BarrelJackMount	Part to hold the barrel Jack	1	PLA (FDM Printer)	—
BasePlate	Laser cut base plate	1	3mm Acrylic	Link
BaseV3	Central Component	1	Resin (MSLA Printer)	—
Bearing	Bearings	24	Steel	Link
Binding Head Screw M3x5	Screws to affix SpoolV2 to Servo Horn	2	Steel	—
BlueBall	For Localization	1	PLA (FDM Printer)	—
CenterRingV3	2 Bodies, Structure & Shell. Print separately. Holes drilled to 2mm for Pins	1	Resin (MSLA Printer)	—
CentreSupport	3D Printed Center Support	1	PLA (FDM Printer)	—
ConnectingBracketV3	Connecting Rods Between Flower Petal and Center Ring. Holes Drilled to 2mm for Pins	5	Resin (MSLA Printer)	—
ESP-32	Microcontroller	1	Electrical Component	Link
ElectronicsPlate	Laser Cut plate that Electronics mount to	1	3mm Acrylic	—

Continued on next page

Table 8 – continued from previous page

Part Name	Description	Qty	Material	Source
FlowerPetal	Flower Petals, Holes drilled to 2mm	5	Resin (MSLA Printer)	—
HeatSetInsert-M2.5x2.5	To affix ServoPlate to ServoTensioner	2	Brass	—
HeatSetInsert-M2.5x4	Insert for Spring	1	Brass	—
HeatSetInsert-M2.5x6	Inserts for Mounting	3	Brass	—
HeatSetInsert-M3x8	Heat Set Insert for Servo Tensioner	1	Brass	—
JST1.5 3 Pin Connector	Male and Female for connector for LED string	1	Electrical Component	—
JST1.5 6 pin connector male	Connector for N20 Motor	1	Electrical Component	—
L293D H bridge Motor Driver	Motor Driver for N20 Motor	1	Electrical Component	Link
M2.5 Nut	Nut to affix Spring	21	Steel	—
M2.5 Nylon Washer	Nylon Washers For Spring and ServoTensioner Affixment	39	Nylon	Link
M3 Nut	Nuts to Fasten Servo Motor	11	Steel	—
M3 Nylon Washer	Nylon Washers for Servo Affixment	22	Nylon	—
M6x6 Heat Set Insert	Placed in CenterRingV3	1	Brass	—
MicroGearmotor v3	300RPM 6V N20 GearMotor	1	Brass, Soft Yellow	—
MidBearingMount-Bottom	Plate for middle sections	3	3mm Acrylic	—
MidBearingMount-Mid	Plate that has bearing pin slots for middle section	1	3mm Acrylic	—
MidPlate	Lower Assembly Plate	1	3mm Acrylic	—
MidPlateShell	Outer Acrylic	3	3mm Acrylic	—
MidPlateShellBearing	Outer Acrylic with bearing shaft holes	1	3mm Acrylic	—

Continued on next page

Table 8 – continued from previous page

Part Name	Description	Qty	Material	Source
OuterShell	Case For Flower Pot	1	PLA (FDM Printer)	—
PCA9685	Servo Motor Controller	1	Electrical Component	Link
Perfboard	Mounts Motor Driver and ESP32 and gives pinouts/ins for components	1	Perfboard	—
Pin	Cut 2x8mm Stainless Steel Rod. Attaches Connecting Rod to both Petals and CenterRing	10	Steel	—
Pinhole Camera	Ultra Tiny USB Webcam Camera with GC0307 Sensor	2	Electrical Component	Link
RedBall	For Localization	1	PLA (FDM Printer)	—
Rollers	Horizontal bearing sleeves	8	PLA (FDM Printer)	—
ServoHorn	Metal Circular Servo Horn Included with Servos	1	Steel	—
ServoMotorBrushless	Servo Motor	1	Electrical Component	Link
ServoPlate	Plates that Servos Mount on	1	3mm Acrylic	—
ServoTensioner	Component that allows for the assembly to be tensioned	1	PLA (FDM Printer)	—
SideSupportWire	Support to hold Servo Assemblies → 1x needed for wire passthrough	1	PLA (FDM Printer)	—
SideSupports	Support to hold Servo Assemblies	1	PLA (FDM Printer)	—
SmallBearingSleeve	Vertical Bearing Sleeves	16	Steel	—
Socket Head Screw M1.6x4	N20 Motor Screws	2	Steel	—
Socket Head Screw M2.5x4	Screw to affix Spring	1	Steel	—

Continued on next page

Table 8 – continued from previous page

Part Name	Description	Qty	Material	Source
Socket Head Screw M2.5x6	Screws to Fasten Servo Tensioner and Spring	3	Steel	—
Socket Head Screw M2.5x8	misc screws	19	Steel	—
Socket Head Screw M3x12	Screws to Fasten Servo Motor	4	Steel	—
Socket Head Screw M3x20	Inner Mid Plate Screws	3	Steel	—
Socket Head Screw M3x25	Outer Assembly Screws	4	Steel	—
Socket Head Screw M3x30	Tensioning Screws	4	Steel	—
Socket Head Screw M3x5	Screw to affix ServoHorn to ServoMotor	1	Steel	—
SpoolV2	Wire Spool	1	Resin (MSLA Printer)	—
Spring-4mmx10	Spring for Tensioning	1	Steel	—
Stainless Steel Cable, 1x7, 0.021"	Tendons. Each approx 1.5ft.	8	Steel	—
Stainless Steel Washer - M3	Steel Washers for Servo Affixment	4	Steel	—
Standoff m2.5x24mm	Flower Pot Legs	4	Plastic	—
Standoff m2.5x8mm	Standoff between base plate and electronics plate	1	3mm Acrylic	—
ThreadedRod	Lead screw for gearmotor	1	Resin (MSLA Printer)	—
TopPlate	Top Plate	1	3mm Acrylic	—
ws2812 5050 led	LED's for the Flower Petals	5	Electrical Component	Link

B CAD & Fabrication Source Files

This appendix indexes where the flower’s design files lives, under **Construction/**: the Fusion 360 models and the print-ready laser-cut sketches that the BOM (Appendix A) and the Assembly Instructions reference.

B.1 Fusion 360 Models

Under **Construction/Fusion360Export/**, split into two folders that mirror the same category structure (**Bases**, **FlowerHeads**, **Stems**, **FullAssemblys**):

- **Current/**: The latest revision of each category: **BaseV7**, **FlowerHeadV3.3**, **ContinuumStemV4**, and the combined **FullBuild2** assembly (**.f3d/.step/.stl**).
- **Archive/**: Prior revision (**BaseV0** through **BaseV6**, etc.), plus a **ToleranceTests** folder of small printed fit-test parts.

B.2 Laser-Cut Sketches

Under **Construction/LaserCutSketches/**: print-ready SVG cut files, designed for **3 mm acrylic sheet**.

Cut convention used in every file: **red** = engrave, **black** = cut, **blue** = deep engrave.

Table 9: Laser-cut SVG files.

File	Contents
LaserBasePlate.svg, LaserMidPlate.svg	Main chassis plates.
LaserTopPlate.svg, LaserTopPlateTopSide.svg	Top plate: main cut file and its top-side engrave detail.
LaserMidPlateShell+ LaserMidBearingMountBottom.svg	Combined sheet: MidPlateShell pieces plus MidBearingMount-Bottom pieces.
LaserMidPlateShellBearing+ LaserMidBearingMountMid-Bearing.svg	Combined sheet: MidPlateShellBearing plus MidBearingMount-Mid (filename kept exactly as exported).
LaserElectronicsPlate.svg	Electronics Plate (Section 1.3.3).
LaserServoMount.svg	ServoPlate, servo mounting bracket.
8mmRodHelper.svg, 10mmRodHelper.svg, 18mmRodHelper.svg	Rod length jigs for cutting the corresponding rod lengths.

Several plates are combined onto shared sheets for cutting efficiency, so one SVG file may correspond to more than one named part in Table 7.

B.3 Bill of Materials Source

Construction/BOM_Aug_18_2026.xlsx is the source spreadsheet Table 7 and Table 8 were exported from.